

1. FITBUDDY – AI FITNESS PLAN GENERATOR

Project Description

FitBuddy is a web-based application that uses AI to generate personalized workout plans and nutrition tips based on fitness goals such as weight loss, muscle gain, or general wellness. It is built with FastAPI and integrated with Google Gemini AI models.

The application accepts user information such as name, age, weight, fitness goal, and preferred workout intensity. It generates a customized 7-day fitness plan and provides nutrition or recovery guidance. Users can submit feedback to update and refine their plans.

SQLite with SQLAlchemy is used for data persistence. The application provides both API endpoints and an interactive HTML interface.

Main Scenarios

1. Generate a personalized 7-day workout plan from user details and intensity.
2. Update an existing plan using feedback such as “more focus on cardio” or “include more rest days.”
3. Provide nutrition or recovery tips based on the selected goal.
4. Allow an admin or coach to view registered users and their original and updated plans.

Core Technologies

FastAPI, Python, Google Gemini, HTML, CSS, Jinja2, SQLAlchemy, SQLite and Uvicorn.

2. PREREQUISITES & PROJECT WORKFLOW

Pre-requisites

1. FastAPI Framework knowledge
2. Gemini API familiarity
3. HTML, CSS and Jinja2 template skills
4. Python programming proficiency
5. Git version control knowledge
6. SQLAlchemy and SQLite basics
7. Python environment setup using pip, virtualenv or conda
8. Uvicorn ASGI server knowledge

Project Workflow

Milestone 1 – Model Selection and Architecture: Select the appropriate generative AI model, define application architecture and set up the development environment.

Milestone 2 – Core Functionalities: Develop workout generation, nutrition tips, feedback updates and FastAPI backend processing.

Milestone 3 – routes.py Development: Build the main routing and application logic connecting the frontend, Gemini services and database.

Milestone 4 – Frontend Development: Design the user interface and create dynamic Jinja2 templates.

Milestone 5 – Deployment: Prepare and test the application for local deployment.

3. MODEL SELECTION & APPLICATION ARCHITECTURE

Activity 1.1 – Model Selection

FitBuddy requires an AI model that can generate structured 7-day workout plans, concise nutrition tips, feedback-based updates and real-time web responses.

Several Hugging Face models were initially explored, including T5, BART, DistilBERT and Bloom. These showed potential but required hosting, GPU resources, fine-tuning or complex prompting for structured plans.

Final Model Selection

- Gemini 1.5 Pro – structured 7-day workout generation and feedback-based plan updating.
- Gemini Flash – fast and practical nutrition or recovery tips.

Reasons for Choosing Gemini

- API-based and easy to integrate with FastAPI.
- Understands structured formatting such as days and sections.
- Requires minimal prompt tuning.
- Supports fast, scalable feedback-based revisions.

Architecture

Frontend: HTML + Jinja2 templates.

Backend: FastAPI for routes and application logic.

AI Layer: Google Gemini Pro and Flash APIs.

Database: SQLite with SQLAlchemy.

The frontend contains index.html, result.html and all_users.html. The backend receives form data, calls Gemini, stores information and renders dynamic templates.

4. DEVELOPMENT ENVIRONMENT & CORE FUNCTIONALITIES

Development Environment

Python and pip are installed first. A virtual environment is created using:

```
python -m venv fitbuddy-env  
fitbuddy-env\Scripts\activate
```

Required libraries include FastAPI, Uvicorn, Jinja2, SQLAlchemy, python-multipart and google-generativeai.

The development server is started with:

```
uvicorn app.main:app --reload
```

The application can be accessed at 127.0.0.1:8000 and API documentation at /docs.

Project Structure

/app – Python files

/templates – HTML templates

/static/images – assets

Core Functionality

Workout plan generation uses **generate_workout_gemini()** in `gemini_generator.py`. User goal and intensity are sent to Gemini 1.5 Pro. The generated plan contains warm-up, main workout with exercises/sets/repetitions, and cooldown or recovery guidance.

Nutrition generation uses **generate_nutrition_tip_with_flash()** in `gemini_flash_generator.py` to provide concise hydration, protein or recovery advice.

5. FEEDBACK, DATABASE & ADMIN FEATURES

Feedback-Based Plan Updating

The `update_workout_plan()` function in `updated_plan.py` lets users revise their plans. Original plan and feedback are sent to Gemini 1.5 Pro, which returns a modified plan. The updated plan and new nutrition tip are then displayed to the user.

User and Plan Storage

Functions include `save_user()`, `save_plan()`, `update_plan()`, `get_original_plan()` and `get_user()`. The `database.py` module stores user information and AI-generated plans.

User information includes ID, name, age, weight, goal and intensity. SQLite and SQLAlchemy ORM are used for persistence. Feedback updates are stored separately so that original and updated plans remain available.

Admin View

Route: `/view-all-users`

Template: `all_users.html`

The admin dashboard displays registered users and their plans. It can show username, age, weight, goal, intensity, original workout plan and updated workout plan. Jinja2 loops are used to render user records in a structured table.

6. FASTAPI BACKEND & ROUTING

Activity 2.2 – FastAPI Backend

All routing logic is handled in `routes.py`. User information is collected through an HTML form and FastAPI `Form(...)` parameters. Pydantic schemas such as `UserInput` and `FeedbackRequest` validate the input structure.

Main Routes

1. `/` – Displays the user form through `index.html`.
2. `/generate-workout` – Processes user input and generates the workout and nutrition tip.
3. `/submit-feedback` – Retrieves the original plan and updates it using user feedback.
4. `/view-all-users` – Displays the administrative dashboard.

`/generate-workout` Processing

The route receives user data, constructs a `UserInput` model, calls `generate_workout_gemini()` and `generate_nutrition_tip_with_flash()`, saves the user and plan, and returns `result.html` with dynamic data.

`/submit-feedback` Processing

The route captures user ID and feedback, retrieves the original plan, calls `update_workout_plan()`, updates the database and renders the revised result.

The `routes.py` module acts as the operational bridge between frontend templates, Gemini AI modules and SQLite database services.

7. FRONTEND DEVELOPMENT & JINJA2

Milestone 4 – Frontend Development

FitBuddy uses HTML, CSS and Jinja2 to create a responsive and interactive interface. The design allows users to submit information, receive personalized plans, provide feedback and allows administrators to manage stored data.

index.html

The main input page captures username, user ID, age, weight, fitness goal and intensity. The design uses a gym-themed background, Google Roboto font, bold typography and a focused layout.

Responsive CSS

Flexbox centers content vertically and horizontally. Media queries support mobile responsiveness. Buttons include hover effects, while input and result areas use padding, shadows and rounded corners.

result.html

Displays the 7-day AI-generated workout plan, nutrition tip and feedback form.

all_users.html

Provides the admin panel for viewing and managing users and their original/updated plans.

Jinja2 Integration

FastAPI's Jinja2Templates is used to render dynamic pages. Backend values such as username, goal, intensity, workout_plan and nutrition_tip are inserted into templates using expressions such as {{ username }} and {{ workout_plan }}.

8. DEPLOYMENT & TESTING

Milestone 5 – Deployment

FitBuddy is prepared for local deployment using FastAPI and Uvicorn. HTML templates are rendered with Jinja2 and AI services are powered by Gemini.

Activity 5.1 – Local Deployment

A virtual environment isolates project dependencies. Required packages are installed from requirements.txt, including FastAPI, Uvicorn, SQLAlchemy and the Google Generative AI SDK.

Example commands:

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Environment Variables

Sensitive information such as the Gemini API key is stored through environment variables or a .env file.

GOOGLE_API_KEY=your_gemini_api_key_here

Activity 5.2 – Testing

The FastAPI development server is started with Uvicorn. The application is accessed at <http://127.0.0.1:8000>. Interactive API documentation is available at /docs. Testing verifies that input collection, AI generation, database storage, feedback updates and template rendering work correctly.

9. APPLICATION PAGES & USER EXPERIENCE

Home Page

The homepage provides a clean interface for entering Name, User ID, Age, Weight, Fitness Goal and Workout Intensity. The Generate Plan button triggers the AI-powered backend.

Personalized Workout Generation Page

After submission, the application displays the user's Name, User ID, Age, Weight, Fitness Goal and Intensity. This information guides the AI engine in producing relevant recommendations.

Workout Plan

FitBuddy creates a day-wise schedule tailored to the user's goal, experience level and preferred intensity. Each day can contain a focus such as full body, upper body, lower body, cardio, core or flexibility. Exercises include sets, repetitions or duration and suggested rest intervals. Warm-up and cooldown guidance may also be included.

Nutrition Tip

The nutrition section provides concise practical advice, including protein sources such as chicken, fish, beans or Greek yogurt. The tip supports healthy eating habits aligned with fitness goals.

Feedback Page

Users provide their User ID and feedback about effectiveness, difficulty or preferences. The system processes the feedback and generates an updated plan, making the experience adaptive.

10. CONCLUSION & PROJECT SUMMARY

Conclusion

FitBuddy – AI Fitness Plan Generator is a comprehensive web application demonstrating the combination of artificial intelligence and health-tech solutions. It uses Google Gemini 1.5 Pro and Gemini Flash to generate personalized 7-day workout plans and nutrition or recovery tips based on user inputs.

The application is built with FastAPI for backend logic and API handling, SQLAlchemy with SQLite for persistent data storage, and Jinja2 for the responsive frontend. Users submit their details and receive a tailored plan containing warm-ups, main workouts, cooldowns and goal-aligned guidance.

A major feature is the dynamic feedback mechanism. Users can comment on the effectiveness or difficulty of their plan, and the application uses generative AI to update it. This supports ongoing customization and engagement.

The administrative view displays registered users and both original and updated plans, supporting transparency and monitoring for coaches, institutions or developers.

Project Summary

- AI-powered personalized fitness planning
- Gemini Pro for workout generation and plan updates
- Gemini Flash for fast nutrition/recovery tips
- FastAPI backend and API endpoints
- SQLite + SQLAlchemy persistent storage
- HTML/CSS/Jinja2 responsive interface
- Feedback-driven plan refinement
- Administrative user and plan monitoring

Overall, FitBuddy demonstrates how generative AI can be integrated into a practical web application to provide structured, personalized and evolving fitness support.